



Faza 1 - Arhitektura

Arhitektura i projektovanje softvera

Kanbify

Članovi tima:

Aleksandar Gospavić 19095

David Marković 19211

Sadržaj

1. Kontekst i cilj softverskog projekta.....	2
2. Arhitekturni zahtevi.....	2
2.1. Arhitekturno specifični (glavni funkcionalni) zahtevi.....	2
2.2. Atributi kvaliteta (ne-funkcionalni zahtevi).....	3
2.2.1. Skalabilnost.....	3
2.2.2. Konzistentnost podataka.....	3
2.2.3. Responsivnost.....	4
2.2.4. Pouzdanost i dostupnost.....	4
2.2.5. Modifikabilnost i proširivost.....	4
2.2.6. Održavanje.....	4
2.3. Tehnička i poslovna ograničenja.....	4
2.3.1. Tehnička ograničenja.....	4
2.3.2. Poslovna i organizaciona ograničenja.....	4
3. Arhitekturni dizajn.....	5
3.1. Arhitekturni obrasci.....	5
3.1.1. Client-Server obrazac.....	5
3.1.2. MVC (Model-View-Controller) obrazac.....	5
3.1.3. Publish-Subscribe (Pub/Sub) obrazac.....	6
3.1.4. Event-Driven arhitektura.....	6
3.1.5. Repository obrazac.....	6
3.2. Generalna arhitektura.....	7
3.3. Strukturni pogled sistema.....	7
3.4. Bihevioralni pogled sistema.....	8
3.4.1. Poziv za kolaboraciju (Invite user).....	8
3.4.2. Odabir table za rad (Select board).....	9
3.4.3. Pomeranje kartice (Card moved).....	9
4. Izbor aplikacionih okvira i biblioteka.....	10
4.1. Klijentska strana (Frontend).....	10
4.2. Serverska strana (Backend).....	10
4.3. Perzistencija podataka.....	11

1. Kontekst i cilj softverskog projekta

Savremeni softverski timovi i organizacije često rade distribuirano, sa više članova koji istovremeno učestvuju u planiranju, realizaciji i praćenju zadataka. U takvom okruženju postoji potreba za alatima koji omogućavaju kolaborativno upravljanje zadacima u realnom vremenu, uz jasnu vizuelizaciju stanja projekta i transparentnost promena.

Kanban metodologija je jedan od najčešće korišćenih pristupa za organizaciju rada, gde se zadaci predstavljaju kao kartice koje prolaze kroz različite faze (npr. To Do, In Progress, Done). Popularni alati poput Jira ili Trello sistema pokazuju značaj ovakvih rešenja, ali ujedno predstavljaju dobar primer složenih softverskih sistema koji kombinuju:

- višekorisnički rad
- real-time sinhronizaciju
- perzistenciju podataka
- jasnu arhitekturnu strukturu

Cilj softverskog projekta **Kanbify** je razvoj višekorisničke web aplikacije za kolaborativno upravljanje projektima po Kanban principu, koja omogućava više korisnika da istovremeno rade nad istim ili različitim projektima (board-ovima), uz podršku za real-time prikaz izmena.

Sistem treba da omogući:

- kreiranje i upravljanje projektima (board-ovima)
- definisanje kolona i kartica unutar projekta
- pomeranje kartica između kolona i promenu njihovog redosleda
- istovremeni rad više korisnika uz sinhronizaciju promena u realnom vremenu
- evidentiranje istorije izmena nad karticama
- kontrolu prava pristupa i ograničenja u zavisnosti od uloge korisnika

Poseban cilj projekta je demonstracija arhitektonskih koncepata kao što su:

- klijent-server arhitektura
- event-driven komunikacija
- publish/subscribe obrazac
- razdvajanje odgovornosti između komponenti sistema

2. Arhitekturni zahtevi

2.1. Arhitekturno specifični (glavni funkcionalni) zahtevi

Arhitekturno specifični zahtevi predstavljaju one funkcionalnosti sistema koje **direktno utiču na izbor arhitekture, tehnologija i obrazaca**, te se razlikuju od običnih CRUD zahteva.

Glavni arhitekturni funkcionalni zahtevi sistema su:

1. Višekorisnički kolaborativni rad

Sistem mora omogućiti da više korisnika istovremeno radi nad istim projektom (Kanban board-om), pri čemu su promene jednog korisnika vidljive ostalim korisnicima u realnom vremenu.

2. Real-time sinhronizacija stanja

Sve izmene nad karticama i kolonama (pomeranje, promena redosleda, dodavanje, izmena) moraju biti propagirane ostalim klijentima bez potrebe za osvežavanjem stranice.

3. Podrška za rad nad više projekata i korisničkih grupa

Sistem mora omogućiti paralelan rad više disjunktih grupa korisnika na različitim projektima, bez međusobnog uticaja.

4. Kontrola konkurentnog pristupa

Sistem mora obezbediti mehanizme za kontrolu istovremenih izmena (npr. zaključavanje projekta ili definisanje korisnika sa pravom izmene), kako bi se sprečile nekonzistentnosti podataka.

5. Perzistencija stanja i istorije izmena

Sve promene nad elementima sistema (kartice, kolone) moraju biti trajno sačuvane u bazi podataka, uz evidentiranje istorije izmena radi auditovanja i praćenja aktivnosti.

6. Asinhrona obrada događaja

Izmene u sistemu moraju generisati domenske događaje koji se obrađuju asinhrono, čime se omogućava razdvajanje poslovne logike, perzistencije i notifikacija.

2.2. Atributi kvaliteta (ne-funkcionalni zahtevi)

Atributi kvaliteta definišu kako sistem treba da se ponaša i kakve karakteristike treba da poseduje, nezavisno od konkretnih funkcionalnosti.

2.2.1. Skalabilnost

Sistem treba da podrži povećanje broja korisnika i projekata bez značajnih promena u arhitekturi. Korišćenje asinhronne komunikacije i Pub/Sub obrasca omogućava lakše horizontalno proširenje sistema.

2.2.2. Konzistentnost podataka

Svi korisnici koji rade nad istim projektom moraju u svakom trenutku imati konzistentan prikaz stanja. Baza podataka predstavlja centralni izvor istine, dok se real-time mehanizmi koriste za sinhronizaciju klijentskih prikaza.

2.2.3. Responsivnost

Korisnički interfejs mora reagovati odmah na korisničke akcije. Ovaj zahtev se ostvaruje korišćenjem optimističkog ažuriranja korisničkog interfejsa, uz kasniju potvrdu sa serverske strane.

2.2.4. Pouzdanost i dostupnost

Sistem mora nastaviti sa radom i u slučaju privremenih grešaka u asinhronim komponentama. Korišćenjem message broker-a omogućava se retry mehanizam i sprečava gubitak događaja.

2.2.5. Modifikabilnost i proširivost

Arhitektura sistema treba da omogući jednostavno dodavanje novih tipova događaja (npr. notifikacije, analitika, audit) bez izmene postojećih komponenti sistema.

2.2.6. Održavanje

Jasno razdvajanje slojeva sistema (klijent, server, komunikacija, perzistencija) omogućava lakše testiranje, održavanje i razumevanje sistema.

2.3. Tehnička i poslovna ograničenja

2.3.1. Tehnička ograničenja

- Sistem mora biti realizovan kao **web aplikacija** dostupna putem standardnog web pregledača, bez potrebe za dodatnom instalacijom na klijentskoj strani.
- Arhitektura sistema mora biti zasnovana na **klijent-server modelu**, sa jasno razdvojenim klijentskim i serverskim komponentama.
- Sistem mora podržavati **real-time komunikaciju** između klijenta i servera radi sinhronizacije stanja između više korisnika.
- Sistem mora koristiti **mehanizam asinhronne razmene poruka** (message broker) za obradu domenskih događaja i razdvajanje odgovornosti između komponenti sistema.
- Podaci sistema moraju biti **trajno perzistirani** u bazi podataka, pri čemu baza predstavlja centralni izvor istine.

2.3.2. Poslovna i organizaciona ograničenja

- Projekat se realizuje u okviru akademskog kursa.
- Razvoj se obavlja u **dvočlanom timu**.
- Vreme za razvoj je ograničeno rokovima definisanim u okviru predmeta.
- Fokus projekta je na demonstraciji arhitektonskih principa i obrazaca, a ne na potpunoj produkcionoj optimizaciji sistema.
- Arhitekturni dizajn

3. Arhitekturni dizajn

3.1. Arhitekturni obrasci

U cilju obezbeđivanja modularnosti, skalabilnosti i podrške za istovremeni rad više korisnika, arhitektura sistema je projektovana korišćenjem sledećih arhitekturnih obrazaca:

3.1.1. Client-Server obrazac

Sistem je zasnovan na **client-server arhitekturi**, gde je klijentska aplikacija odgovorna za prikaz korisničkog interfejsa i slanje korisničkih zahteva, dok serverska aplikacija obrađuje poslovnu logiku, upravlja stanjem sistema i vrši perzistenciju podataka.

Primena u sistemu:

- Klijent (web aplikacija) omogućava korisnicima interakciju sa Kanban tablama
- Server obrađuje zahteve za kreiranje i izmenu projekata, kolona i kartica

Razlog primene:

- Jasno razdvajanje odgovornosti
- Centralizovana kontrola nad podacima
- Pogodno za višekorisničke sisteme

3.1.2. MVC (Model–View–Controller) obrazac

Serverska aplikacija koristi **MVC obrazac** radi organizacije koda i razdvajanja slojeva odgovornosti.

Primena u sistemu:

- *Controller* prima HTTP zahteve i validira ulazne podatke
- *Service/Model* sloj sadrži poslovnu logiku i pravila sistema.
- *View* je implicitno realizovan kroz JSON odgovore ka klijentu.

Razlog primene:

- Povećana čitljivost i održivost koda
- Lakše testiranje poslovne logike
- Standardan i široko prihvaćen obrazac u web sistemima

3.1.3. Publish–Subscribe (Pub/Sub) obrazac

Za sinhronizaciju stanja između više korisnika koristi se **publish–subscribe obrazac**, zasnovan na razmeni poruka putem message broker-a.

Primena u sistemu:

- Izmene nad karticama (npr. pomeranje između kolona) se publikuju kao događaji
- Više komponenti sistema može reagovati na isti događaj (npr. perzistencija, obaveštavanje drugih korisnika).

Razlog primene:

- Slabo povezivanje komponenti
- Podrška za real-time kolaboraciju
- Lakše proširenje sistema dodatnim funkcionalnostima

3.1.4. Event-Driven arhitektura

Sistem delimično primenjuje **event-driven pristup**, gde se ključne promene u sistemu modeluju kao domenski događaji.

Primena u sistemu:

- Akcije korisnika (kreiranje, izmena ili pomeranje kartice) generišu događaje
- Događaji se asinhrono obrađuju od strane odgovarajućih komponenti.

Razlog primene:

- Prirodno modelovanje poslovnih
- Bolja podrška za asinhronu operacije

3.1.5. Repository obrazac

Za pristup i manipulaciju podacima koristi se **repository obrazac**, koji apstrahuje rad sa bazom podataka od ostatka aplikacije.

Primena u sistemu:

- Poslovna logika ne zavisi direktno od konkretne baze podataka.
- Sav pristup podacima je centralizovan kroz repozitorijume.

Razlog primene:

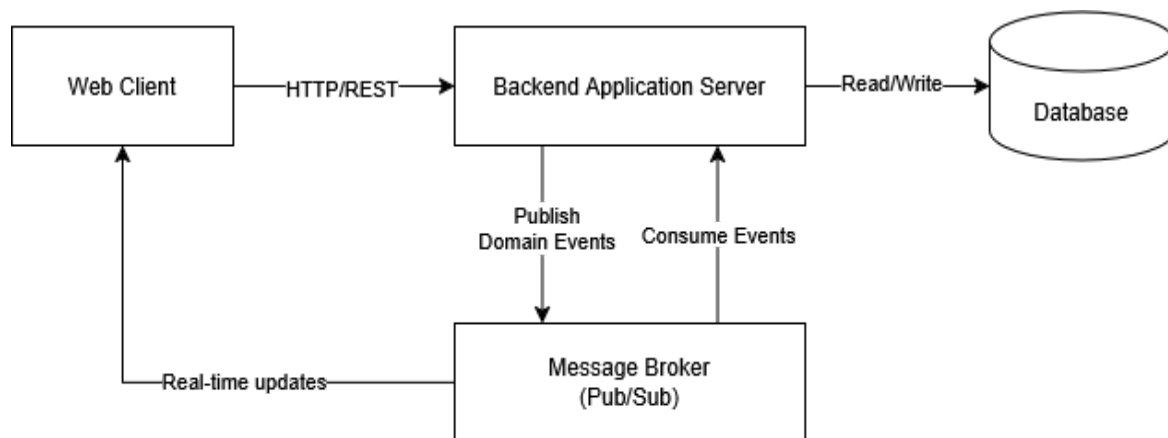
- Lakša promena ili proširenje perzistentnog sloja
- Čistija i testabilnija poslovna logika

3.2. Generalna arhitektura

Sistem je projektovan kao višeslojna web aplikacija zasnovana na client-server arhitekturi. Klijentska aplikacija predstavlja korisnički interfejs sistema i omogućava interakciju korisnika sa Kanban tablama, dok serverska aplikacija sadrži centralizovanu poslovnu logiku i upravlja stanjem sistema.

Za podršku real-time kolaboraciji i sinhronizaciji između više korisnika koristi se mehanizam asinhronne razmene poruka putem message broker-a, zasnovan na publish-subscribe obrascu. Izmene nad domenskim objektima se modeluju kao događaji koji se publikuju i obrađuju nezavisno od glavnog toka obrade zahteva.

Podaci sistema se trajno čuvaju u bazi podataka, koja predstavlja centralni izvor istine. Ovakva arhitektura omogućava jasno razdvajanje odgovornosti između komponenti, lakše održavanje sistema i potencijalno proširenje funkcionalnosti u budućnosti.

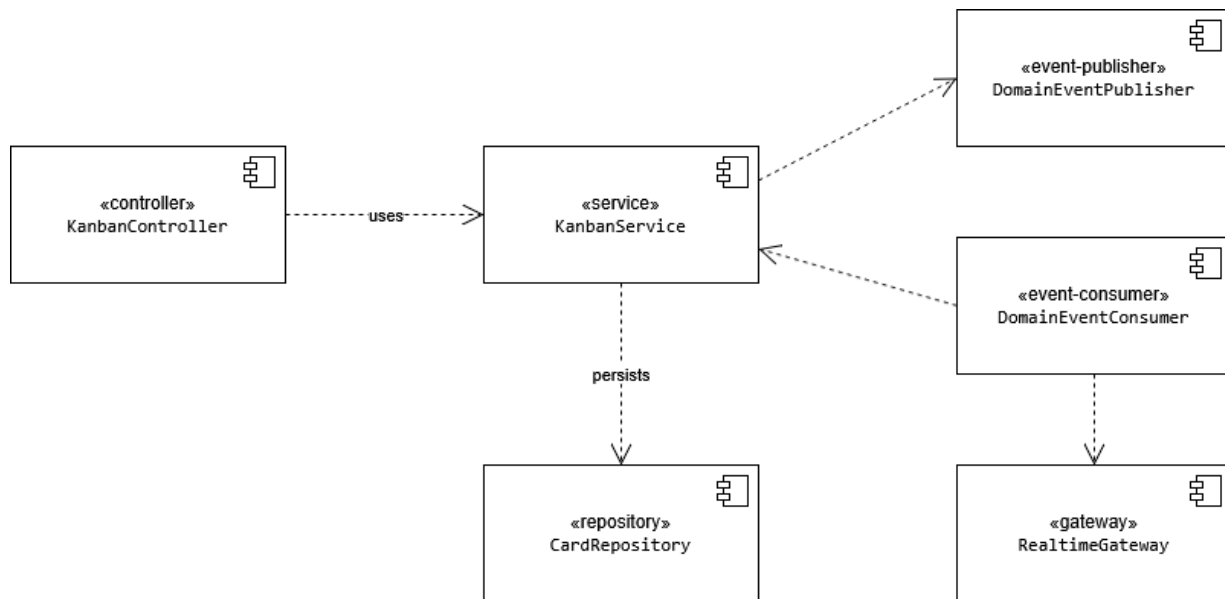


3.3. Strukturni pogled sistema

Strukturni pogled sistema prikazan je putem UML Component dijagrama, koji ilustruje modularnu organizaciju serverske aplikacije. Serverska aplikacija je podeljena na jasno definisane komponente sa razdvojenim odgovornostima, u skladu sa principima slojevite arhitekture.

Kontrolerski sloj je odgovoran za prijem korisničkih zahteva, dok servisni sloj implementira poslovnu logiku sistema. Pristup perzistentnim podacima je apstrahovan putem repository komponente. Događaji nastali kao posledica korisničkih akcija publikuju se putem posebne komponente, dok se njihova obrada

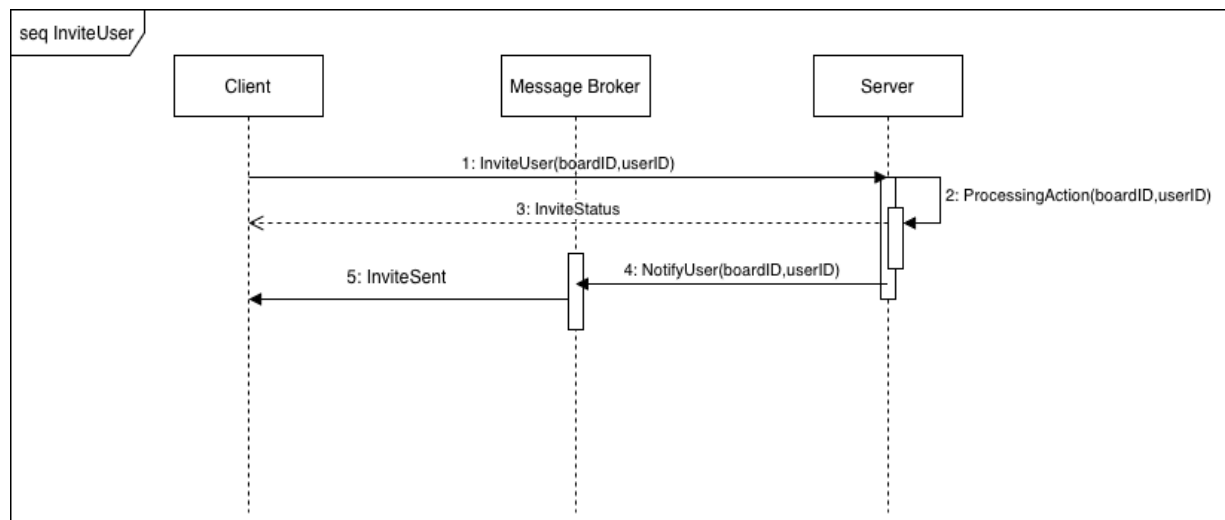
vrši asinhrono kroz potrošače događaja, koji dalje obaveštavaju klijente putem real-time komunikacionog mehanizma.



3.4. Bihevioralni pogled sistema

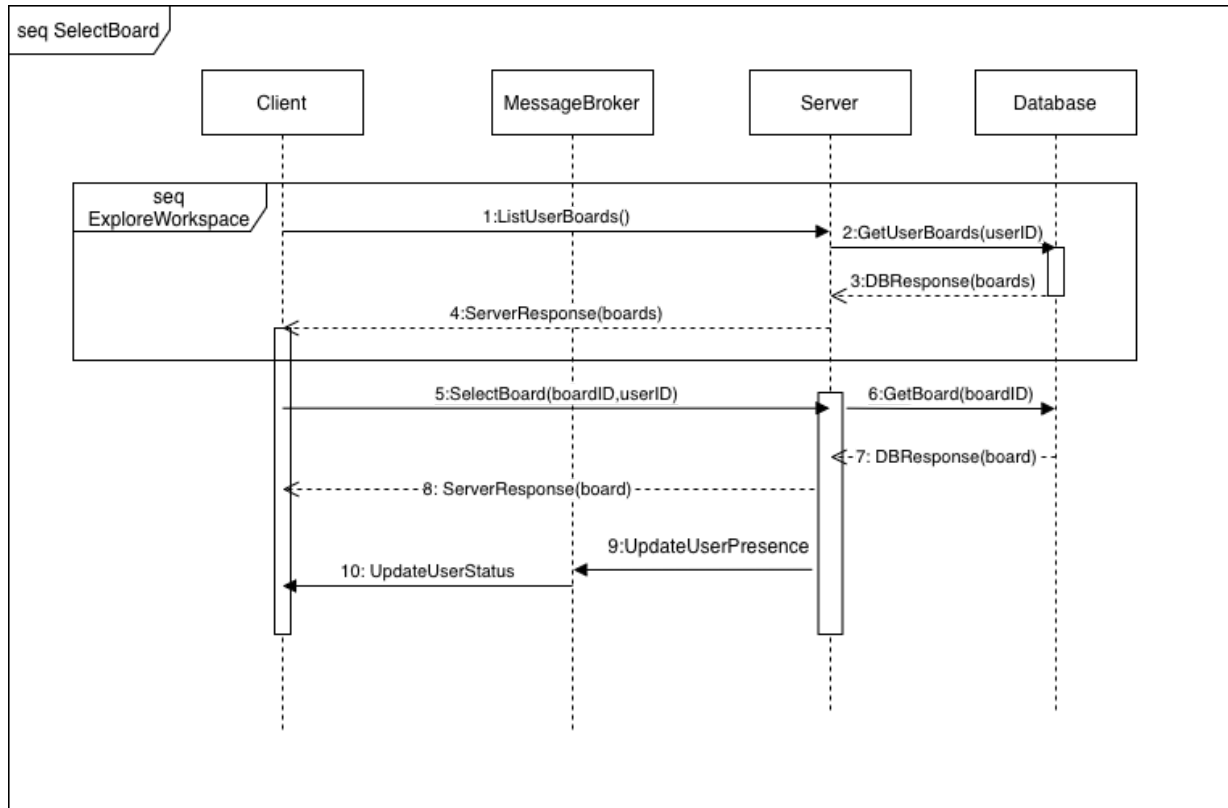
3.4.1. Poziv za kolaboraciju (Invite user)

Dijagram sekvence za slanje pozivnice korisniku na određenu tablu. Prikazuje komunikaciju između klijenta i servera radi obrade zahteva, potvrdu statusa pozivnice klijentu, te korišćenje Message Broker-a za asinhrono slanje obaveštenja o pozivu ciljanom korisniku.



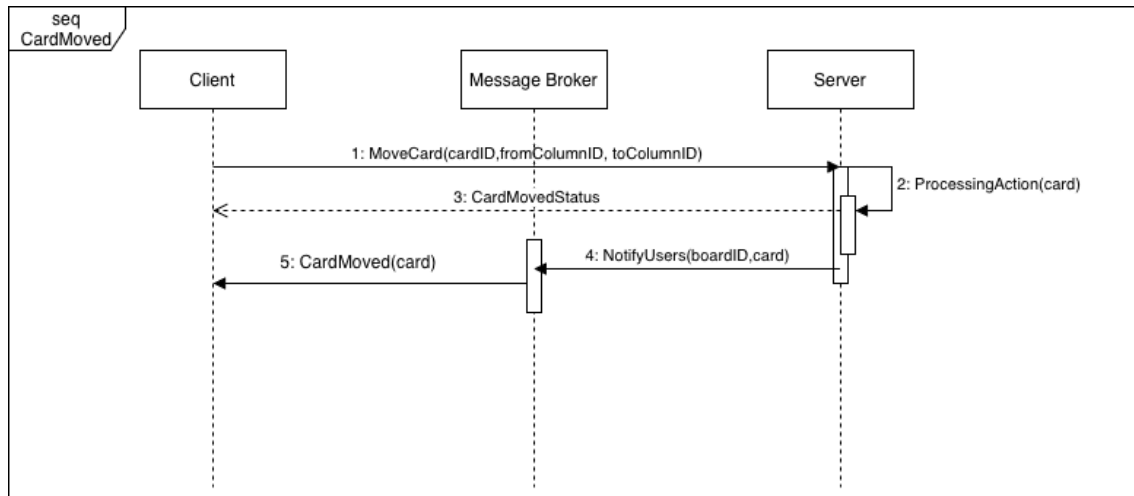
3.4.2. Odabir table za rad (Select board)

Ovaj dijagram prikazuje proces u kojem klijent prvo preuzima listu svih dostupnih tabli iz baze podataka, a zatim vrši odabir specifične table. Nakon učitavanja detalja table, sistem automatski ažurira status prisustva korisnika i šalje obavještenje ostalim učesnicima putem Message Broker-a.



3.4.3. Pomeranje kartice (Card moved)

Ovaj dijagram ilustruje sinhronizaciju u realnom vremenu prilikom pomeranja kartice unutar table. Nakon što klijent inicira promenu, server obrađuje akciju i putem Message Broker-a distribuira informaciju svim ostalim povezanim korisnicima, osiguravajući konzistentan prikaz radnog prostora.



4. Izbor aplikacionih okvira i biblioteka

4.1. Klijentska strana (Frontend)

Za implementaciju klijentskog dela sistema biće korišćen **React.js** kao JavaScript biblioteka za izgradnju korisničkog interfejsa. React omogućava razvoj komponentno orijentisanih, interaktivnih i responzivnih web aplikacija, što je pogodno za realizaciju Kanban table sa velikim brojem dinamičkih elemenata (kolone, kartice, drag-and-drop interakcije).

Na klijentskoj strani se primenjuje **MVC/MVVM koncept**, gde:

- *View* predstavlja React UI komponente,
- *Model* čine stanje aplikacije i podaci dobijeni sa servera,
- *ViewModel/Controller* logika se ostvaruje kroz React hook-ove, state management i pozive ka serverskim API-jima i WebSocket konekciji.

Za real-time prijem promena (npr. pomeranje kartica od strane drugih korisnika) klijentska aplikacija koristi **WebSocket komunikaciju**.

4.2. Serverska strana (Backend)

Serverski deo sistema biće implementiran korišćenjem **NestJS** framework-a, koji je zasnovan na Node.js platformi i podstiče modularnu i slojevit arhitekturu aplikacije.

NestJS prirodno podržava **MVC arhitekturni obrazac**, gde:

- **Controller** sloj obrađuje HTTP zahteve klijenata,
- **Service** sloj sadrži poslovnu logiku,
- **Repository** sloj je zadužen za pristup i perzistenciju podataka.

Za real-time komunikaciju između servera i klijenata koristiće se **NestJS Gateway** mehanizam zasnovan na **WebSocket** tehnologiji, koji omogućava trenutnu distribuciju promena svim povezanim klijentima.

Za internu asinhronu komunikaciju unutar backend aplikacije koristiće se **NestJS EventEmitter**, koji omogućava emitovanje i obradu domenskih događaja (npr. pomeranje kartice, izmena sadržaja, dodela korisnika) bez čvrstog povezivanja komponenti.

4.3. Perzistencija podataka

Za čuvanje podataka koristiće se **PostgreSQL** relacionalna baza podataka, uz upotrebu **TypeORM** biblioteke radi mapiranja domenskih objekata na bazu podataka i pojednostavljenog rada sa perzistencijom.